



Lecture 8: Child Components

INTERVIEW QUESTION

[> CORE](#)

How do you use one component inside another?

Your `App.svelte` is getting crowded. How do you split it into smaller pieces and reference them?

As your web application grows, putting everything into a single file quickly becomes impossible to manage.

The Problem with Monolithic Files

Imagine you are building a dashboard application, and your `App.svelte` file has grown to 1,500 lines of code. It contains the logic for a top navigation bar, a massive collapsible sidebar, a complex interactive data grid, and a footer.

Whenever you need to fix a tiny bug in the sidebar, you have to scroll past hundreds of lines of unrelated data grid logic, desperately searching for the exact `<div>` you need to change. Worse, because all the variables and styles are trapped in the same global scope of that file, a teammate might accidentally change a variable meant for the header and inadvertently break the footer. Your codebase has become a tangled, fragile monolith.

Splitting into Child Components

To solve this, Svelte allows you to break your user interface into smaller, entirely self-contained building blocks called **components**.

Instead of one giant file, you cut the sidebar logic out and paste it into its own file named `Sidebar.svelte`. You do the same for the header in `Header.svelte`. Each component encapsulates its own HTML markup, its own CSS styling, and its own JavaScript logic.

Importing and Rendering

Once you have separated a piece of your interface into its own `.svelte` file, you need a way to bring it back into the main application.

You do this in two simple steps:

1. **Import the file** in your `<script>` tag, just like a standard JavaScript module.
2. **Render it** in your markup as if it were a custom HTML tag.

Svelte components must always be Capitalized when used in markup. This is how the Svelte compiler distinguishes your custom `<Sidebar>` component from a standard lowercase HTML `<div>` or `<span>` .

App.svelte

```
1  <script>
2    import Header from './Header.svelte';
3    import Sidebar from './Sidebar.svelte';
4  </script>
5
6  <div class="layout">
7    <Header />
8
9    <main>
10      <Sidebar />
11      <p>Main dashboard content goes here.</p>
12    </main>
13  </div>
```

Import the child component file

Render it like a custom HTML tag

The Component Tree

When you use one component inside another, you are creating a parent-child relationship. In the example above,

`App.svelte` is the **parent component**, while `Header` and `Sidebar` are its **child components**.

These child components can, in turn, import and render their own children (like a `<NavButton>` inside the `<Sidebar>`), creating a deeply nested Component Tree that perfectly mirrors the visual hierarchy of your application.

KEY TAKEAWAY

Each instance of a component is completely independent. If you render `<Sidebar />` twice on the same page, each one will get its own separate, isolated copy of any internal `$state` variables.

Summary

To prevent your files from becoming unmaintainable monoliths, you should split discrete parts of your UI into separate `.svelte` files. To use these files, you `import` them into the parent component's script block, and then render them in the markup using an uppercase tag. This encapsulation allows you to build complex, scalable applications out of small, reliable, and reusable building blocks.

	HTML TAGS (Native)	SVELTE COMPONENTS (Custom)
Naming	Lowercase (<code><div></code> , <code></code>)	Uppercase (<code><Widget></code> , <code><Sidebar></code>)
Source	Built into the browser natively	Imported from a <code>.svelte</code> file
Behavior	Standard DOM elements	Encapsulated custom logic and styling